FACULTY OF COMPUTER SCIENCE AND MATHEMATICS

BACHELOR OF COMPUTER SCIENCE

(SOFTWARE ENGINEERING) WITH HONOURS

CSE3433 SOFTWARE ARCHITECTURE

SEMESTER II SESSION 25/26

PROJECT TITLE:

GROUP 14

RECRUITX - EVENT-DRIVEN JOB RECRUITMENT & APPLICATION PROCESSING SYSTEM

PHASE 2 : ARCHITECTURE DESIGN

PREPARED FOR:

DR. MOHAMAD NOR BIN HASSAN

PREPARED BY:

NAME	MATRIC NO.
NUR ALIN HAYANI BINTI AHMAD SYUKRI	S75034
AINAA NADHIRAH BINTI ASMADI	S75246
MARYAM MUSFIRAH BINTI ARIFF HAZIZAN	S75922

TABLE OF CONTENT

1. System Overview

RecruitX is a web-based recruitment system designed to improve and automate the hiring process. In many organisations, recruitment is still handled manually, where HR staff review resumes, contact candidates, and track application progress manually. This often leads to delays, inefficiency, and communication issues.

To address these problems, RecruitX provides an automated platform that manages job postings, application submissions, candidate screening, interview scheduling, and status updates. The system reduces repetitive manual work and improves communication between candidates and recruiters.

RecruitX is built using an event-driven approach, where user actions trigger system events that automatically initiate related processes. For example, when a candidate submits an application, the system immediately processes the data, screens the resume, and sends a notification without requiring manual intervention.

The system supports three main users:

- Candidate – applies for jobs and tracks application status
- Recruiter / HR Staff – manages job postings and candidate progress
- System Services – handle automation such as screening, notifications, and analytics

2. Functional Requirements

FR01 - The system shall allow all the recruiters to create and publish the job listings which will trigger a JobPosted event to update the public job board automatically.

FR02 - The system shall, when submission of an application, trigger an ApplicationReceived event to provide the candidate with an intermediate automated confirmation.

FR03 - The system shall trigger an ApplicationStatusChanged event when a recruiter updates all the candidate progress, automatically sending an update to the candidate to eliminate the communication delays.

FR04 - The system shall automatically screen the incoming resumes based on pre-defined keywords like skills or year of experience to assist HR in narrowing down to the applicant pool without manual effort.

FR05 - The system shall provide a centralized interface for recruiters to move the candidate through stages like shortlisted or interviewed, ensuring no qualified candidate is overlooked.

FR06 - The system shall allow the recruiters to initiate the InterviewRequested event which is automatically coordinated with the candidate's availability and sends the calendar invites.

FR07 - The system shall provide a dashboard that monitors the "hiring funnel" in real-time, reducing all the human error in data entry or tracking.

3. Non- Functional Requirements

NFR-01- Availability; The system shall be able to maintain an uptime of 99.5%, ensuring that recruiters and candidates can access the platform at any time including during peak hiring periods.

NFR-02 - Scalability: The system shall be able to handle a sudden surge of up to 1,000 concurrent event messages without a decrease in performance or message loss.

NFR-03 - Reliability: The system shall ensure guaranteed message delivery if an event fails to reach a consumer, the event bus must retry the delivery until successful.

NFR-04 - Performance: The system shall ensure that event triggers occur within 3 seconds of the candidate clicking the “Submit” button.

NFR-05 - Security: The system shall encrypt all sensitive candidate data at rest while it is stored in the database and while it is being send over the network

NFR-06 - Maintainability: The system architecture shall be modularly decoupled, allowing new event consumers to be added without requiring a reboot of the main application.

4. Architecture Style Explanation

RecruitX uses an Event-Driven Architecture (EDA) with a Publish/Subscribe model, where components communicate by producing and consuming events instead of directly interacting.

When a user performs an action (e.g., submitting an application), an event is sent to a central event bus, allowing multiple services to respond automatically. This approach reduces dependency between components and enables efficient automation.

This architecture is suitable for RecruitX as it improves system speed, scalability, and overall efficiency in handling the recruitment process *(GeeksforGeeks, 2024)* .

4.1 Event Producers

- Candidate Portal
    - Publishes events such as ApplicationReceived when a candidate submits a job application
- Recruiter / HR Portal
    - Publishes events like JobPosted, ApplicationStatusChanged, and InterviewRequested when recruiters perform actions

4.2 Event Bus

- Receives events generated by producers
- Routes events to the correct services based on event type
- Acts as a buffer to handle high volumes of events
- Ensures that services remain loosely coupled

4.3 Event Consumers

- Application Management Service
    - Stores application data when an ApplicationReceived event occurs
- Resume Screening Service
    - Automatically evaluates resumes based on predefined criteria

- Notification Service
    - Sends emails or alerts when events like application submission or status updates occur
- Interview Scheduling Service
    - Handles interview requests and sends calendar invitations
- Analytics Service
    - Collects data from events to generate reports and insights

EDA is preferred over traditional architectures because it allows asynchronous processing and better scalability during high user activity.

5. Service / Component Identification

This section describes the modular components of RecruitX and their specific responsibilities within the event-driven ecosystem.

5.1 Production Components:

- User Interface (UI) Module: The front-end component which is Web/Mobile that captures user inputs from candidates and recruiters.
- Event Publisher Module: An internal component in the portals that formats user actions into the standardized event message which is JSON/XML and sends them to the broker.

5.2 Middleware Components

- Event Bus: The main infrastructure responsible for message persistence, queuing and routing logic
- Event Registry: A component that stores the definitions and schemas for every event type

5.3 Processing Components (Consumers)

- Database Manager: A component within the Application Management Service that handles CRUD operations for the main database
- Resume Parser: A specific component within the Screening Service that extract text from PDF for the analysis
- Calendar API integration: A component within the Scheduling Service that synchronizes with the external calendar providers such as Google Calendar.
- Data Aggregator: A component within the Analytics Service that summarizes event logs into meaningful metrics for dashboard.

5.4 Service Decomposition and Boundary

Service Name	Responsibility (What it DOES)	Exclusions & Boundaries (What it DOES NOT do)
Application Management	Responsible for managing the workflow lifecycle of an application and maintaining the profile state of candidates.	Does not perform resume parsing or automated screening; its boundary is strictly limited to application data persistence.
Job Posting Service	Maintains accurate job openings, requirements, and hiring manager data to be displayed on the job board.	Does not manage candidate progress or interview logistics; it is dedicated solely to vacancy metadata.
Screening Service	Processes and evaluates applicant suitability through automated keyword matching and fit score generation.	Does not store the final application record or handle candidate communication; it acts as a standalone analytical engine.
Interview Service	Manages scheduling logistics, interviewer feedback, and provides meeting links via external API integration.	Does not update job listing details or parse resumes; its scope is limited to coordination and calendar synchronization.
Notification Service	Delivering timely updates, automated emails, and alerts to both applicants and HR staff based on	Does not contain business logic for hiring decisions; it functions only as a communication output

	system events.	channel.
Analytics Service	Collects and summarizes event logs into meaningful metrics for real-time hiring funnel dashboards.	Does not modify the status of individual applications or manage job postings; its boundary is restricted to data aggregation and reporting.

6. Architecture Diagrams

6.1 System Context Diagram

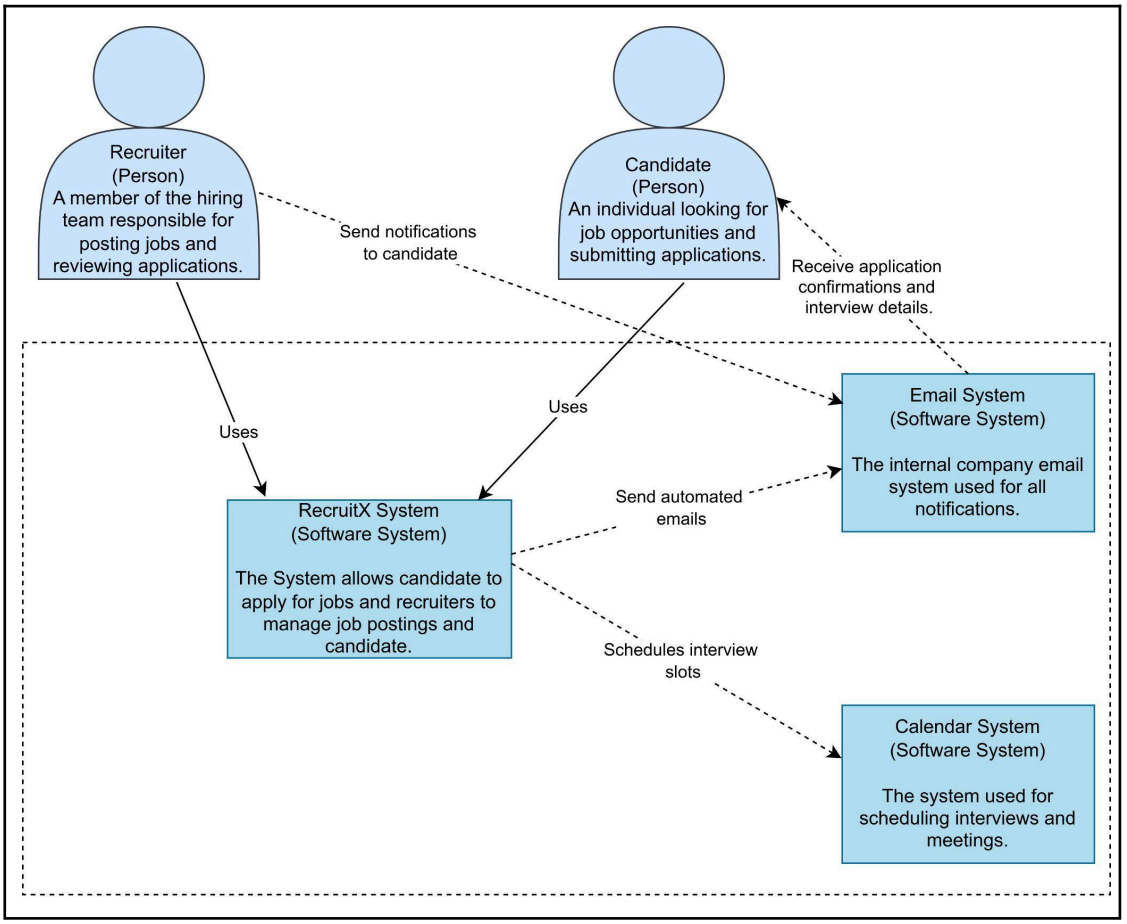

Figure 6.1 System Context Diagram RecruitX System

Figure 6.1 is the System Context Diagram for RecruitX System shows how the RecruitX system interacts with its users and external services at a high level.

In this system, the candidate uses RecruitX to apply for jobs and check their application status. At the same time, the recruiter uses the system to manage job postings and review candidates during the hiring process.

The RecruitX system also connects to external services to support its functions. It sends notifications through the email service, which is used to inform candidates about application updates and interview details. In addition, the system

interacts with a calendar service to schedule interviews between candidates and recruiters.

Overall, the diagram shows that RecruitX acts as the main platform that connects users and external services, helping to make the recruitment process smoother and more organised.

6.2 Component / Service Architecture Diagram

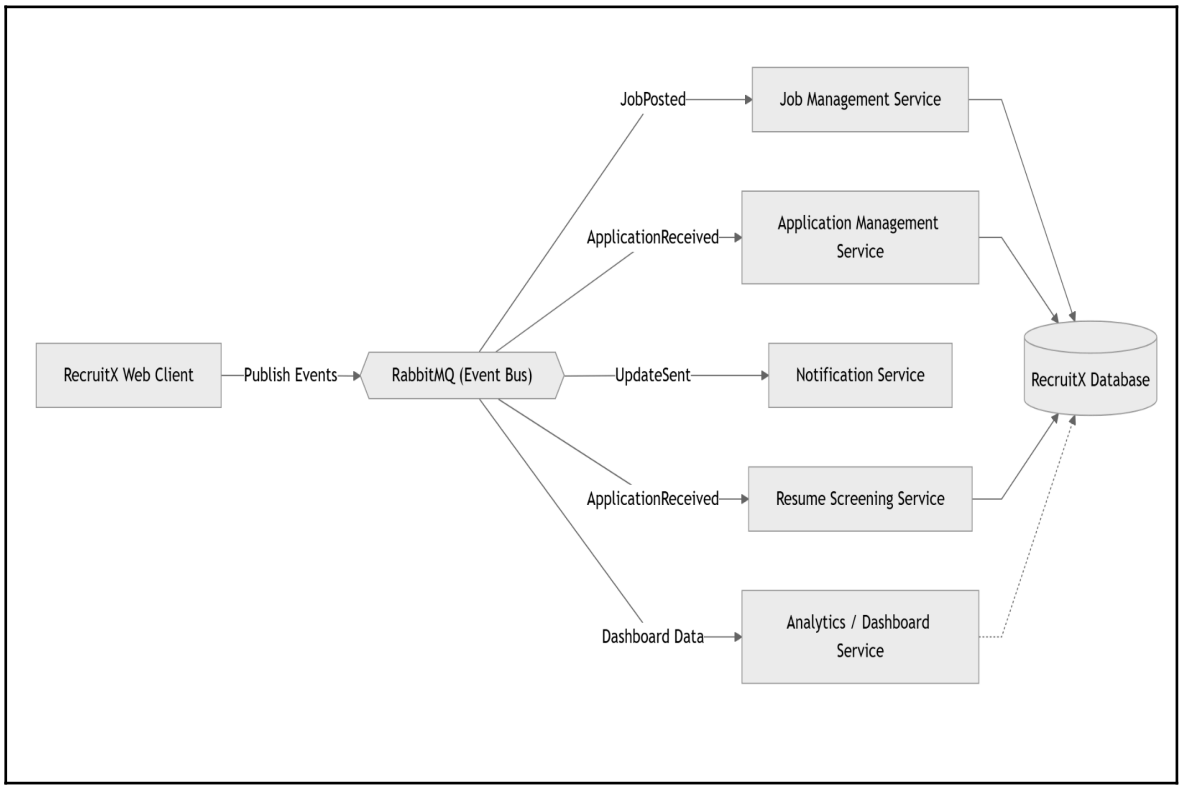

Figure 6.2 Component / Service Architecture Diagram for RecruitX system

This is the component diagram which depicts how the key services work and interact within the RecruitX. In the event-driven architecture, events are propagated through the Event Bus (RabbitMQ), which in turn propagates them to the relevant services like Application processing service, Notification service, and interview scheduling service.

6.3 Interaction Diagram

Candidate Application Interaction Flow

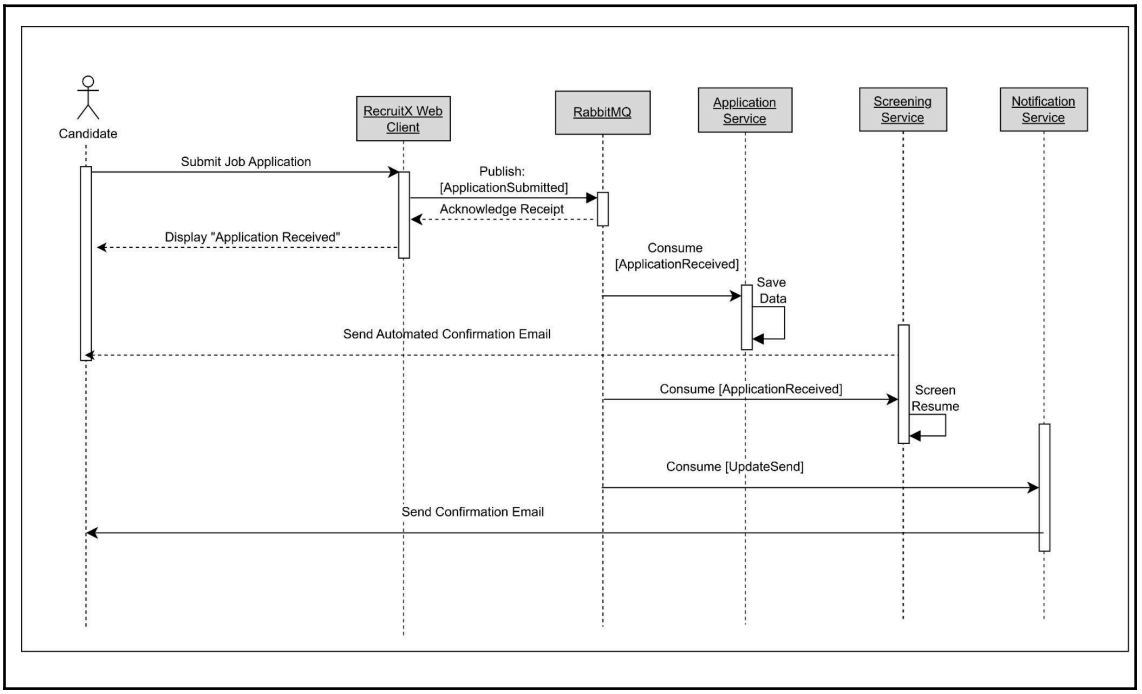

Figure 6.3.1 Candidate Application Interaction Flow for RecruitX system

The Candidate Application flow demonstrates the efficiency of Event-Driven Architecture in the asynchronous processing. When a candidate submits an application, the RecruitX Web Client publishes the ApplicationSubmitted event to RabbitMQ which immediately returns an Acknowledge Receipt. This allows the user to see the “Received” confirmation instantly without waiting for the backend to finish their task for ensuring high system performance.

In the background, RabbitMQ distributes the event to the independent services. The Application Service persists the data, Screening Service evaluates the resume and the Notification Service sends a confirmation email. This loose coupling ensures that if one service fails the other still remain and not affected. By processing these in parallel, the system achieves superior scalability and reliability during high traffic recruitment periods.

Recruiter Interaction Flow

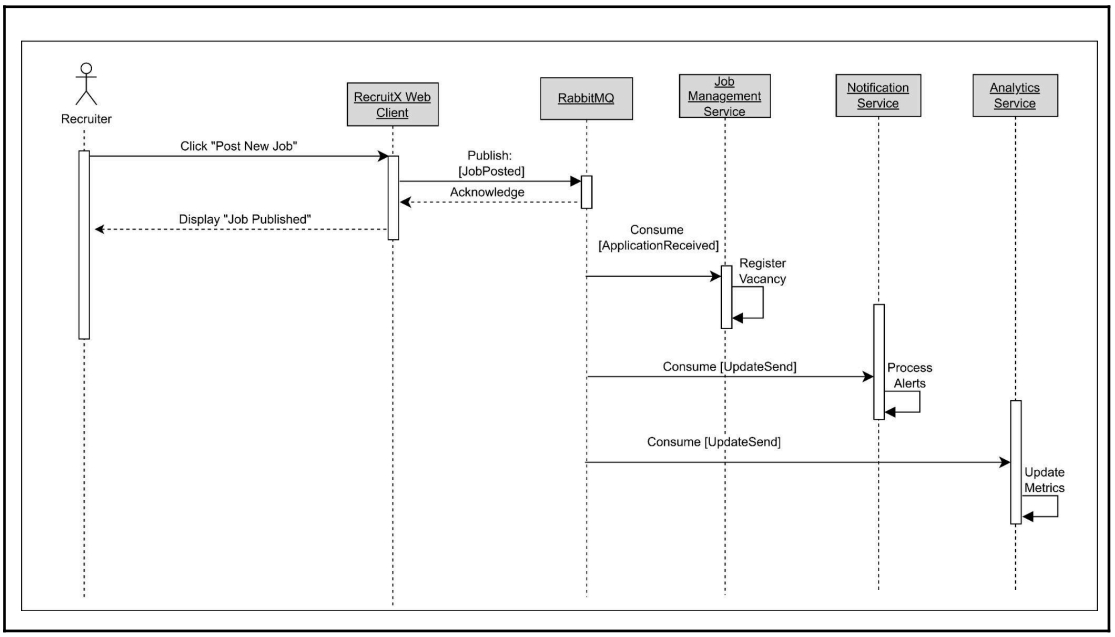

Figure 6.3.2 Recruiter Interaction Flow for RecruitX system

The Recruiter Interaction flow focuses on the publication of a new vacancy. When the Recruiter clicks “Post New Job”, the Web Client publishes a JobPosted to RabbitMQ. The system provides an immediate Acknowledge Receipt to the Recruiter by displaying a “Job Publish” confirmation. This can help Recruiter to continue their work without waiting for the other service to finish their internal processing.

Next, RabbitMQ triggers three concurrent background tasks. The Job Management Service registers the vacancy in the system, while the Notification Service processes alerts to inform relevant candidates. Other than that, the Analytics Service updates system metrics to reflect the new posting on the dashboard. This parallel execution ensures that the platform remains efficient and all the data remain synchronized across all the recruitment modules.

6.4 Deployment Diagram

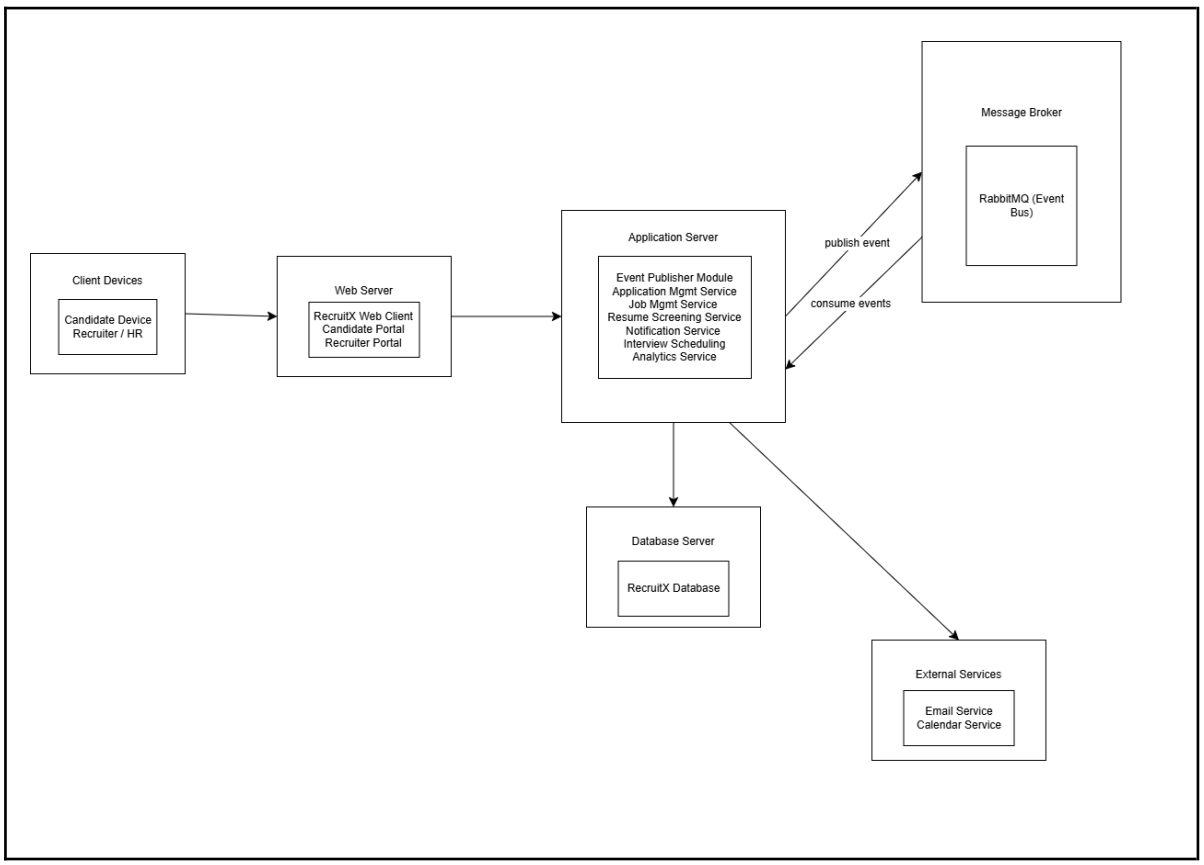

Figure 6.4 : Deployment Diagram for RecruitX systen

The deployment diagram shows how the RecruitX system is organised across different devices and servers. Users (candidates and recruiters) access the system through client devices, which connect to the web server that hosts the RecruitX web client.

The web server forwards user requests to the application server, where the main services such as Application Management, Job Management, Resume Screening, Notification, Interview Scheduling, and Analytics are located.

When an action occurs, the application server publishes events to RabbitMQ, which acts as the message broker. These events are then consumed by the application services to perform tasks such as processing applications and sending notifications.

The application server also interacts with the database server to store and retrieve system data. In addition, it connects to external services such as email and calendar services for notifications and interview scheduling.

Overall, this deployment supports the event-driven architecture by separating system components and allowing efficient communication through the event bus.

7. Data Ownership

In an Event-Driven architecture for job recruitment and application processing, data ownership is decentralized following the “Database per Service” pattern, each microservice owns the data it produces and is responsible for managing its own state, ensuring loose coupling and scalability.

Service / Component	Data Owned	Event Produced	Responsibility
Applicant Service	Applicant Profile, Resume, Portfolio, Contact Info.	ApplicantRegistered, ProfileUpdated	Managing applicant personal data & profile state.
Job Posting Service	Job Details, Requirements, Hiring Manager Data.	JobPosted, JobClosed, JobUpdated	Maintaining accurate job openings & requirements.
Application Service	Application State (Draft, Submitted), Submission Timestamp.	ApplicationSubmitted, AppStatusChanged	Managing the workflow lifecycle of an application.
Screening Service	Assessment Scores, Resume Keyword Match, Fit Score.	ScreeningPassed, ScreeningFailed	Processing and evaluating applicant suitability.
Interview Service	Interview Schedule, Interviewer Feedback, Meeting Links.	InterviewScheduled, FeedbackSubmitted	Managing scheduling and evaluation logistics.
Notification	Email Templates,	EmailSent,	Delivering timely

Service	Notification Logs.	NotificationFailed	updates to applicants and staff.
Analytics Service	Aggregated Pipeline Metrics, Time-to-hire, Source Effectiveness.	AnalyticsUpdated	Processing events for business intelligence.

8. References

GeeksforGeeks. (2024, January 31). *EventDriven Architecture System Design*. GeeksforGeeks. https://www.geeksforgeeks.org/system-design/event-driven-architecture-system-design/

Bhargava. (2025, July 30). *Mastering Event-Driven Architecture for Bulletproof Modern Applications (Part 2)*. Medium. https://medium.com/@bhargavkoya56/mastering-event-driven-architecture-for-bulletproof-modern-applications-part-2-c971d8b3ed4b

Sharma, L. (2022). *10 nonfunctional requirements to consider in your enterprise architecture*. Redhat.com. https://www.redhat.com/en/blog/nonfunctional-requirements-architecture